Postgres Table Partitioning

pruning, retention, and no sharding

Watch the 3-min video

In one line: when one table is huge but the whole database still fits on one server, split that table into partitions inside Postgres: queries that filter on the partition key skip the rest (pruning), indexes stay small, and old data is removed by dropping a partition instead of a giant DELETE.

1 The three-billion-row audit log

Your `audit_log` table has **3 billion rows** and grows by **10 million a day**. Queries for last week crawl, the indexes no longer fit in memory, and deleting old rows takes hours and bloats the table.

You don't need more servers. You need to **split this one table, inside the same database**. That is table partitioning.

2 How declarative partitioning works

In Postgres you declare a **parent table**, partitioned by a key, and attach **child tables** called partitions. Your app still queries the parent as one table. Postgres **routes each inserted row** to the right partition, and **reads only the partitions a query needs**. It all happens on one server, with **no sharding**: no cross-node joins, no distributed transactions.

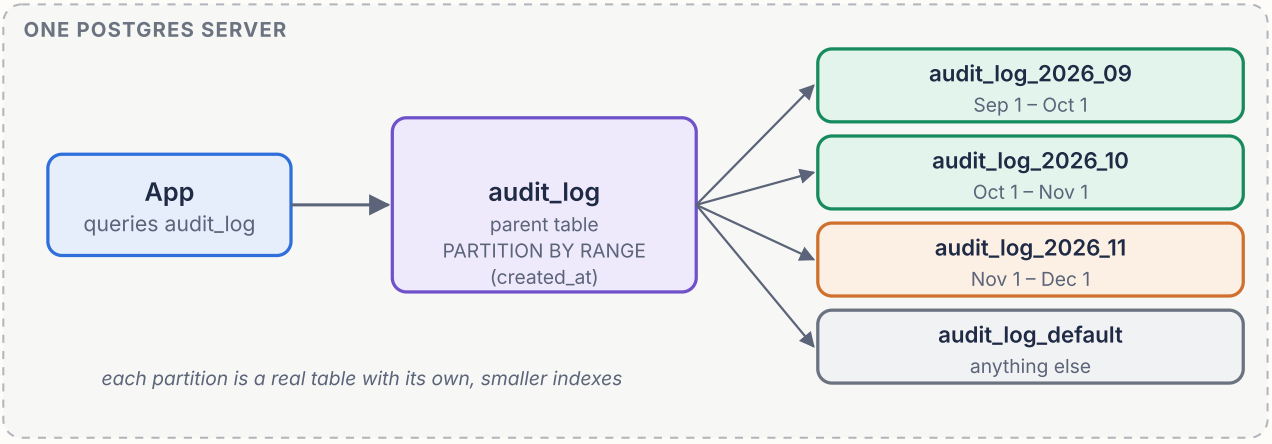


Figure 1. The application only knows audit_log. Postgres stores the rows in monthly child tables and a default partition.

3 Three strategies

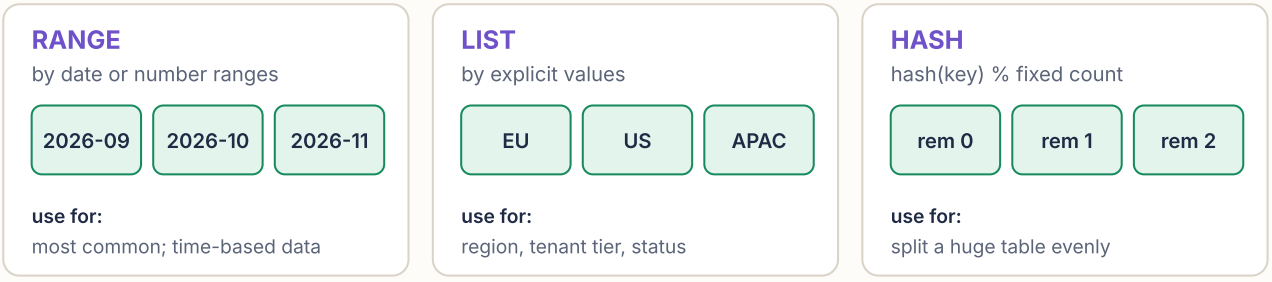


Figure 2. Range is the usual choice for logs, events and orders; list groups by known values; hash spreads rows evenly when there is no natural range.

Here is the audit log, partitioned by range on `created_at`, one partition per month. Each partition gets its own, smaller indexes. Note that ranges are **inclusive at the start and exclusive at the end**, so October is *from Oct 1 up to Nov 1*, and no row falls between two months.

```
CREATE TABLE audit_log (
  id          bigint GENERATED ALWAYS AS IDENTITY,
  created_at  timestamptz NOT NULL,
  actor_id    bigint,
  action      text,
  PRIMARY KEY (id, created_at)          -- must include the partition key
) PARTITION BY RANGE (created_at);

CREATE TABLE audit_log_2026_10 PARTITION OF audit_log
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');

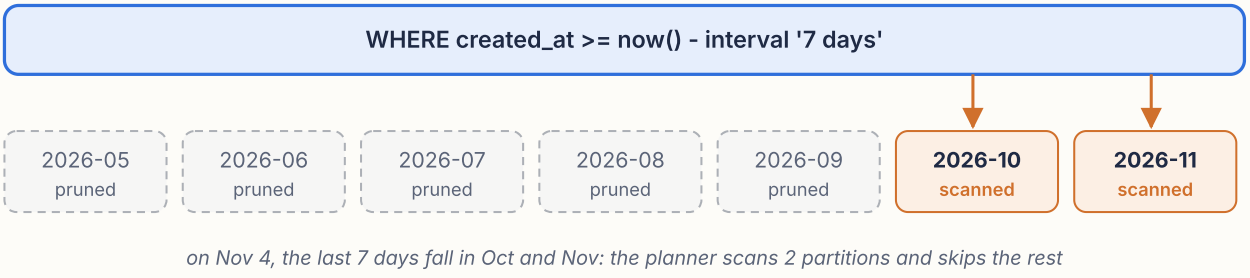
CREATE TABLE audit_log_default PARTITION OF audit_log DEFAULT;

CREATE INDEX ON audit_log (actor_id, created_at);  -- created on every partition
```

A **default partition** catches rows that match no range, so an insert never fails just because next month's partition is missing. But don't rely on it: rows piling up there lose the benefits of pruning.

4 Win one: partition pruning

A query that filters on `created_at`, for the last seven days, only scans the partitions that can contain those rows. The planner **skips every other month**. You can see it in `EXPLAIN`: only the matching partitions appear in the plan.



on Nov 4, the last 7 days fall in Oct and Nov: the planner scans 2 partitions and skips the rest

Figure 3. Pruning turns a scan over 3 billion rows into a scan over the last month or two (the EXPLAIN below filters from Nov 1, so only one partition remains).

```
EXPLAIN SELECT * FROM audit_log
 WHERE created_at >= '2026-11-01' AND actor_id = 42;

 Index Scan using audit_log_2026_11_actor_id_created_at_idx
   on audit_log_2026_11        -- the only partition in the plan
```

The flip side: a query **without the partition key** in its filter, such as `WHERE actor_id = 42` alone, must visit **every** partition.

5 Win two: cheap retention

To delete data older than a year, you don't run a giant DELETE. You **detach the old partition and drop it**, in seconds, with no bloat and no heavy vacuum. You can also archive the partition to cheap storage (for example, dump it to object storage) before dropping it.

```
ALTER TABLE audit_log DETACH PARTITION audit_log_2025_10;
-- optional: pg_dump -t audit_log_2025_10 ... > archive (then upload)
DROP TABLE audit_log_2025_10;
```

WORKED EXAMPLE: REMOVING ONE MONTH

- At 10 million rows a day, one month is about **300 million rows**.
- **DELETE ... WHERE created_at < ...** marks 300 million rows dead one by one, writes all of it to the WAL, then leaves vacuum to clean up for hours. The table and its indexes stay bloated meanwhile.
- **DETACH + DROP** removes the partition's files: a metadata change that takes seconds, whatever the row count.
- Last-week queries now touch one or two monthly partitions of ~300M rows each, instead of 3 billion, and each partition's index is a fraction of the size.

6 Pitfalls

- **Keys must include the partition key.** A primary key or unique constraint is enforced per partition, so it must contain `created_at`: `PRIMARY KEY (id, created_at)`, not `(id)` alone.
- **Not too many partitions.** Thousands of partitions slow down query planning and add overhead. Monthly for years of data is fine; per-hour for a decade is not.
- **Create future partitions ahead of time.** They must exist before data arrives. Automate it with the `pg_partman` extension or a scheduled job that creates the next few months.

COMMON MISTAKES

- Partitioning a table whose main queries don't filter on the partition key: every query scans every partition.
- Forgetting to create next month's partition, so rows pile up in the default partition (or inserts fail without one).
- Confusing partitioning with sharding: it is still one server, so it doesn't add CPU, disk or write capacity.
- Still deleting old rows with DELETE after partitioning, instead of dropping partitions.

7 Key terms

Table partitioning: one logical table split into child tables on one server

Range / list / hash: the three Postgres strategies

DETACH PARTITION: turns a partition into a normal table

Parent table: what the app queries; holds no rows itself

Default partition: catches rows no range matches

pg_partman: extension that creates and drops partitions

Partition: child table with a slice of the rows

Partition pruning: planner skips partitions that can't match

Sharding: splitting data across servers (not this)

8 Interview questions

Q1. When would you partition a table without sharding? BASIC

When one table is huge but total data and load still fit on one server, and most queries filter by a key like time. You get pruning, smaller indexes and cheap retention, without the complexity of sharding.

Q2. What is partition pruning? BASIC

The planner uses the WHERE clause on the partition key to skip partitions that can't contain matching rows. A last-7-days query on monthly partitions scans one or two partitions, not all of them.

Q3. Name the three partitioning strategies in Postgres. BASIC

Range (dates, numbers; most common), list (explicit values like EU, US, APAC), and hash (hash of the key modulo a fixed number of partitions).

Q4. Your events table keeps 1 year of data at 10M rows/day. How do you handle retention? SCENARIO

Range-partition by month. Each month, detach and drop (or archive, then drop) the partition older than 12 months. That removes ~300M rows in seconds, with no DELETE, no bloat and no long vacuum.

Q5. After partitioning, a dashboard query got slower. Why? SCENARIO

It probably doesn't filter on the partition key, so it visits every partition (and plans for each). Add a created_at filter, or add an index the query can use on each partition.

Q6. Why does Postgres reject PRIMARY KEY (id) on a table partitioned by created_at? SCENARIO

Uniqueness is enforced by per-partition indexes, so Postgres can only guarantee it if the key includes the partition column. Use PRIMARY KEY (id, created_at).

Q7. Does partitioning help write throughput? TRICKY

Not much. It's still one server, one WAL and one disk. It helps by keeping the hot partition's indexes small and in memory, but to scale writes beyond one machine you need sharding.

Q8. Why not use daily partitions for 10 years of data? TRICKY

That's about 3,650 partitions. Planning time and per-partition overhead grow, and queries without a tight time filter touch thousands of tables. Monthly (120) is a better fit; daily only if you keep a few months.

Next: SD-E-027 · Distributed ID Generation backendroadmap.com